Synchronization

Distributed Algorithms

- Algorithms that are intended to work in a distributed environment
- Used to achieve tasks such as:

- Communication
- Accessing resources
- Allocating resources
- etc.

- Synchronization and Coordination linked to distributed algorithms

- Achieved using distributed algorithms

Why Synchronize?

- It is important to *control access* to a single, shared resource and agree on the *ordering of events* such as whether message *m1* from process *P* was sent before or after message *m2* from process *Q.*
- Synchronization in Distributed Systems is much more difficult than in uniprocessor systems.
- How processes cooperate and synchronize with each other?

Synchronization And Coordination

- making sure that processes doing the right thing at the right time.

- Two fundamental issues:
- Coordination (the right thing)
- Synchronization (the right time)

- allowing processes to synchronize and coordinate their actions

Synchronization

- Synchronization is coordination with respect to time, and refers to the **ordering** of events and execution of instructions in time.
- “Ordering of all actions”

- Total ordering of events
  (whether message m1 from process P was sent before or after message m2 from process Q)
- Ordering of access to resources
- Examples of synchronization - Ordering distributed events and ensuring that a process performs an action at a particular time.

Coordination

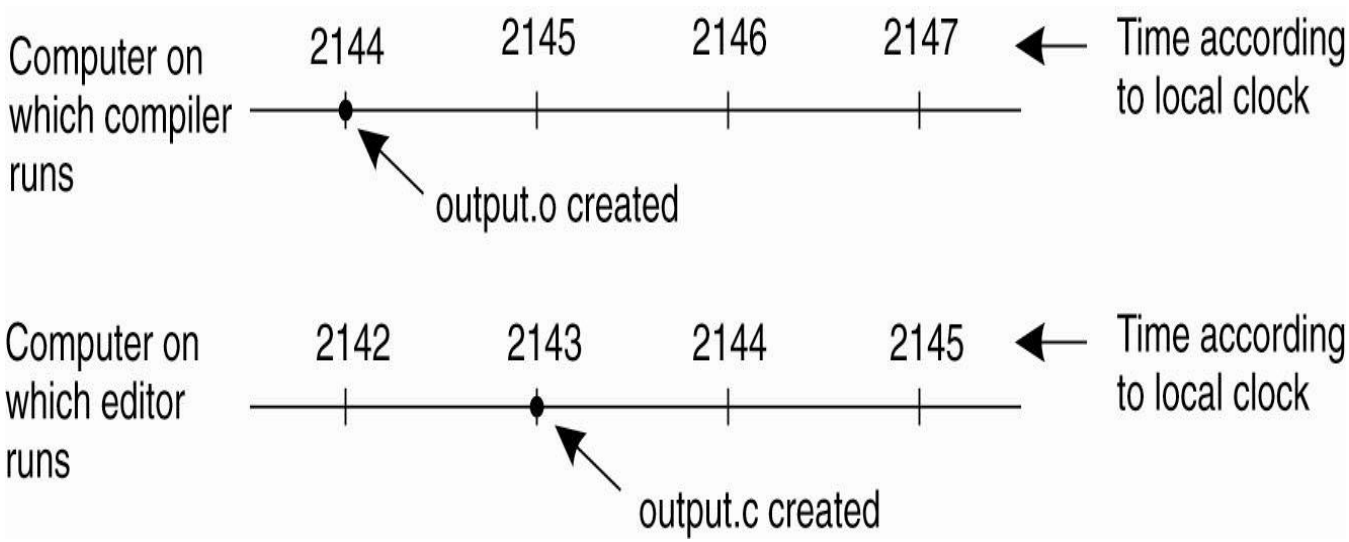
- Coordination refers to coordinating the actions of separate processes relative to each other and allowing them to agree on global state.
- "**Coordinate actions and agree on values.**"

- Coordinate Actions:
  - What actions will occur
  - Who will perform actions
- Agree on Values:
  - Agree on global value
  - Agree on environment
  - Agree on state
- Examples of coordination include ensuring that processes agree on –
  - what actions will be performed?
  - who will be performing actions?
  - state of the system?

Main Issues

- Time and Clocks: synchronizing clocks and using time in distributed algorithms
- Global State: how to acquire knowledge of the system's global state
- Concurrency Control: coordinating concurrent access to resources
- Coordination: when do processes need to coordinate and how do they do it

Clock Synchronization



For make command in Unix/Linux: When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time

Clock Synchronization

- Synchronization based on “Actual Time” is really easy on a uniprocessor system.
- Achieving agreement on time in a DS is not trivial.
- Is it even possible to synchronize all the clocks in a Distributed System?
- Skew- Disagreement on reading of two clocks
- Drift – Difference in the rate at which two clocks count the time(due to crystal material, heat, voltage, humidity, etc.)
- With multiple computers, “clock skew” ensures that no two machines have the same value for the “current time”. But, how do we measure time?

How Do We Measure Time?

- Turns out that we have only been measuring time accurately with a "global" atomic clock since *Jan. 1st, 1958* (the "beginning of time").
- Measuring time is not as easy as one might think it should be.
- Algorithms based on the current time (from some **Physical Clock**) have been devised for use within a DS.

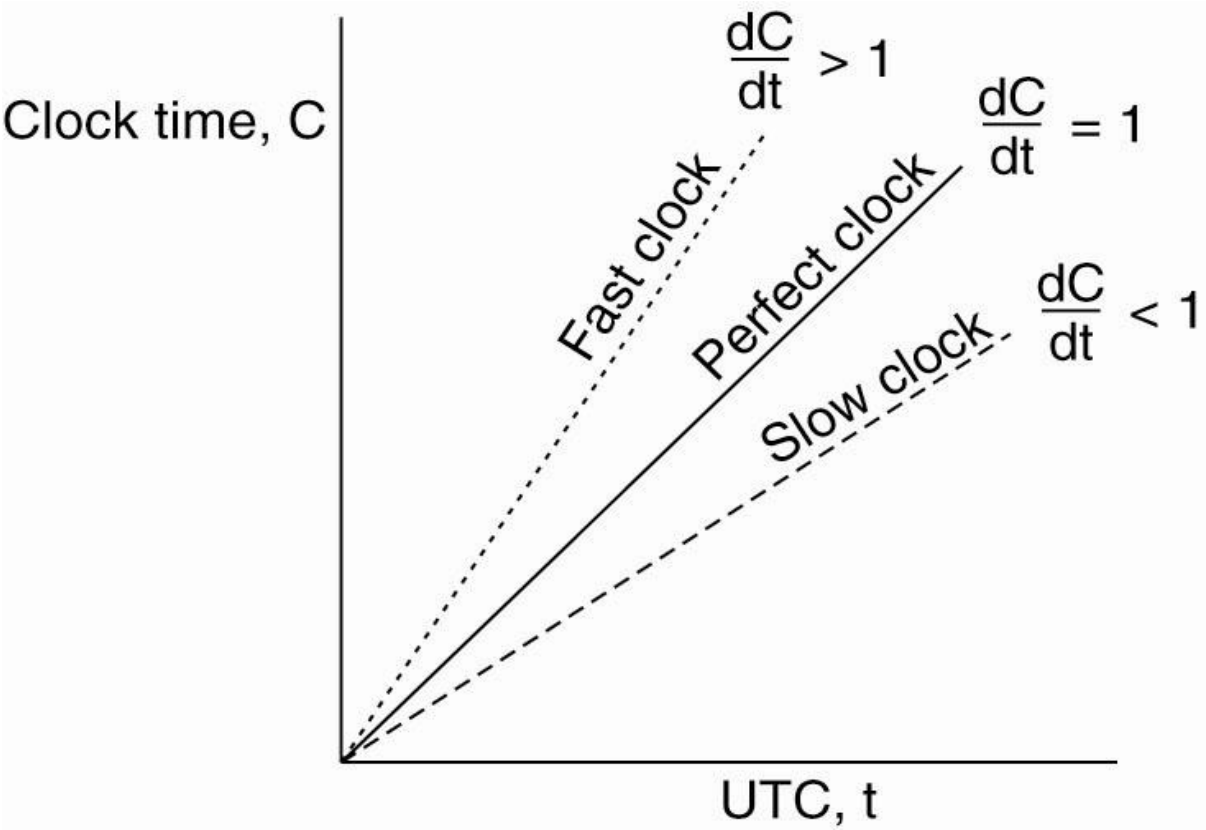
Clock synchronization algorithms

- UTC – Universal Coordinated Time
- When UTC time is t, the value of the clock on machine p is Cp(t) = t (in perfect world)
- That means dC/dt=1
- If there exists some constant q,

$1-q <= dC/dt <= 1+q$

Where q is maximum drift rate

Clock synchronization algorithms

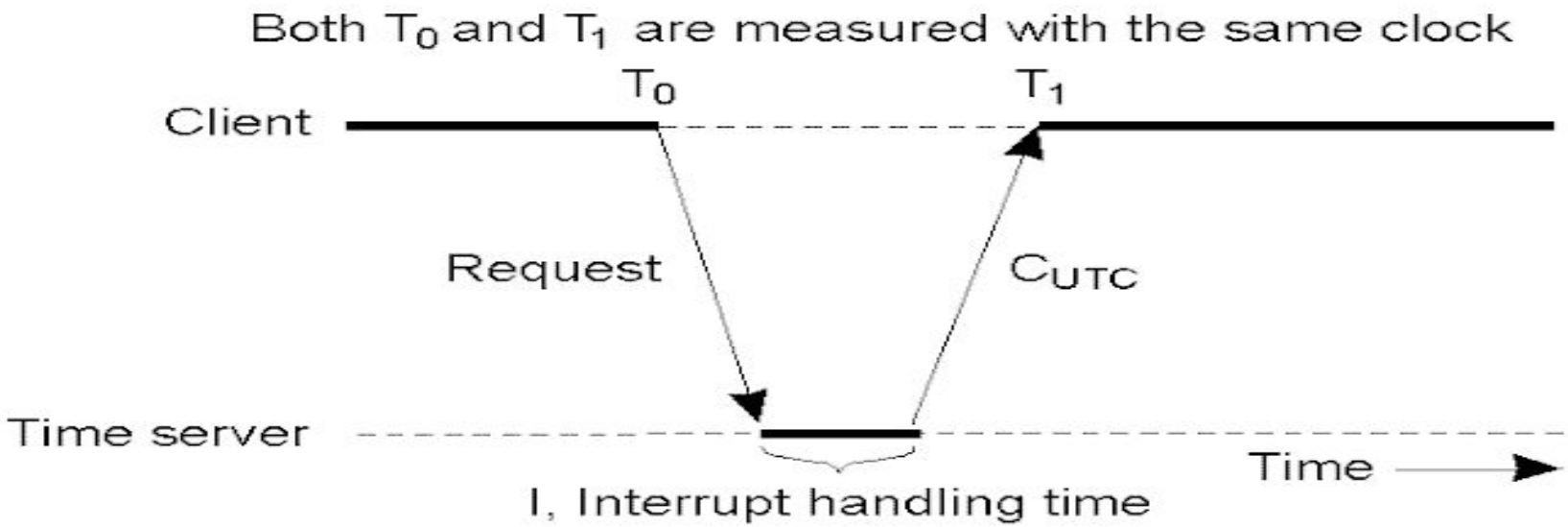


The relation between clock time and UTC when clocks tick at different rates(slow, perfect, fast)

Clock synchronization algorithms

- There exists a *time server* receiving signals from a UTC source
  - Cristian's algorithm
- There is no UTC source available
  - Berkley's algorithm
- Exact time *does not matter!*
  - Lamport's algorithm

Cristian's Algorithm



- Cristian's algorithm requires clients to periodically synchronize with a central time server (typically a server with a UTC receiver)
- Every computer periodically asks the "time server" for the current time
- The server responds ASAP with the current time C_{UTC}
- The client sets its clock to C_{UTC}

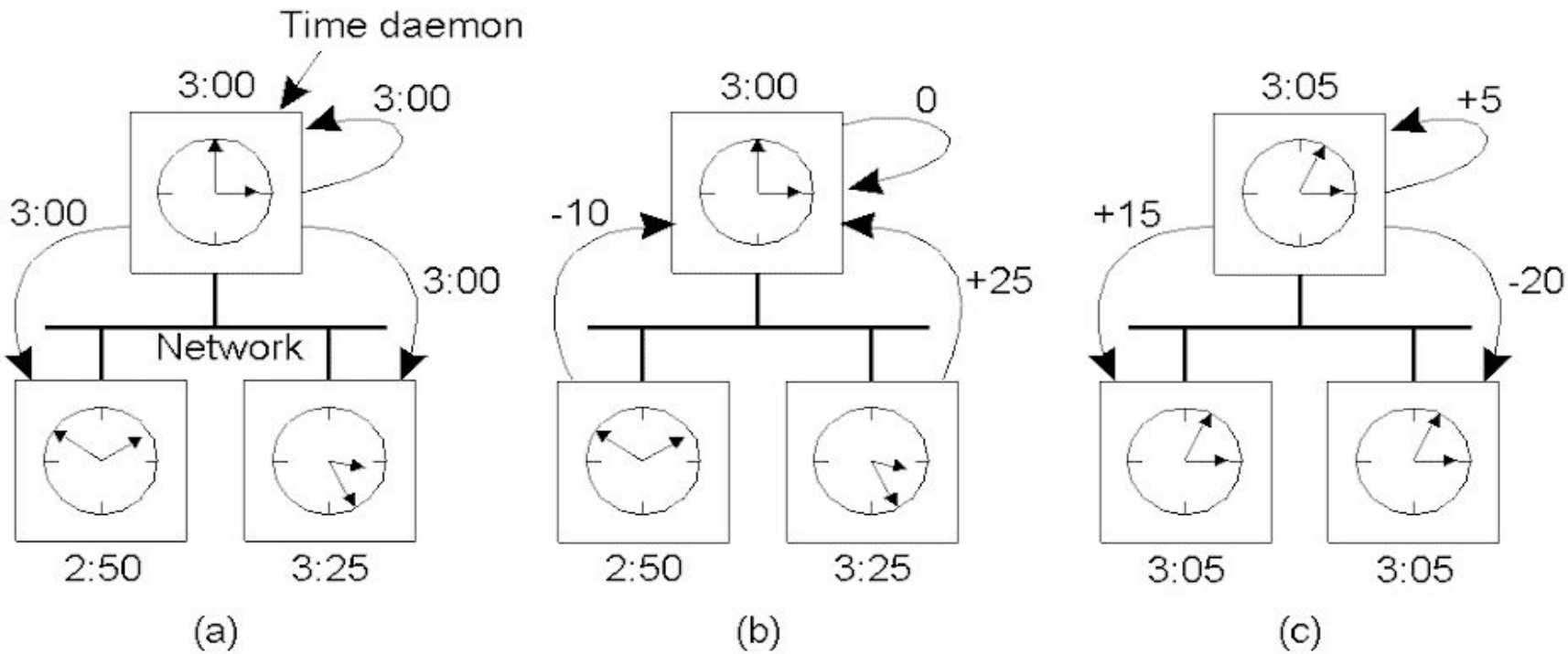
Cristian's Algorithm

- **Major problem -** if time from time server is less than the client – resulting in time running backwards on the client! (Which cannot happen – time does not go backwards).
- **Minor problem** - results from the delay introduced by the network request/response latency.
- Estimate message propagation time – (T1-T0)/2
- One way propagation time – (T1-T0-I)/2
- To improve accuracy – take series of measurements and discard those whose T1-T0 exceeds threshold value

Berkeley Algorithm

- In Christian's algorithm – Time server is passive
- An algorithm for internal synchronization of a group of computers
- A *master* polls to collect clock values from the others (*slaves*)
- The master uses round trip times to estimate the slaves' clock values
- It takes an average
- It sends the required adjustment to the slaves
- If master fails, can elect a new master to take over

The Berkeley Algorithm



a) The time daemon asks all the other machines for their clock values

b) The machines answer

c) Takes an average & tells everyone how to adjust their clock

Berkeley Algorithm

Computers	Clock Reading
A (daemon)	3:00
B (left)	2:50
C (right)	3:25

Computers	Ahead/Behind
A (daemon)	0:00
B (left)	-0:10
C (right)	+0:25

Berkeley Algorithm

- Average time: 3:05

Computers	Needed Adjustment
A (daemon)	+0:05
B (left)	+0:15
C (right)	-0:20

Other Clock Sync. Algorithms

- Both Cristian's and the Berkeley Algorithm are centralized algorithms.
- Decentralized algorithms also exist- Averaging algorithms, Internet's Network Time Protocol (NTP)
- NTP achieves worldwide accuracy in the range of 1-50 msec through the use of advanced clock synchronization algorithms.
- Multiple external time sources
- Use of synchronized clocks – using timestamp

  - G = CurrentTime – MaxLifeTime – MaxClockSkew

  - G is global variable, MaxLifeTime is max time a message can live, and MaxClockSkew is how far from UTC the clock might be at worst

  - any timestamp older than G can safely be removed

Averaging algorithms

- Divide time into fixed-length resynchronization intervals
- ith interval starts at T0+iR and runs until T0+(i+1)R, where T0 is an agreed-upon moment in the past, R is a system parameter
- At the beginning of each interval, every machine broadcasts the current time as per its clock
- After broadcast, each machine will start a local timer to collect all other broadcasts arrive during some interval S
- Average the values received from others
- Variation 1 – discard highest and lowest values and average the rest
- Variation 2 – correct each message by adding estimate propagation time from the source